



DOCUMENTO 1: GUÍA DEL TALLER PRÁCTICO



TALLER PRÁCTICO SEMANA 8: Programación Asíncrona en Vue 3

De la prueba manual (Postman) a la integración automatizada (Vue 3 + Axios)

Evaluación: 10% Individual + 5% Grupal (Total 15%)

Duración: 90-120 minutos

Enfoque: Investigativo, pensamiento crítico y buenas prácticas de ingeniería de software.



Continuidad con la Semana 7

En la **Semana 7**, aprendiste a probar los endpoints de tu servidor Express (**mi-servidor**) utilizando Postman. Verificaste que **GET /api/movimientos** y **POST /api/movimientos** funcionaran correctamente. En esta **Semana 8**, daremos el siguiente paso: **conectaremos el frontend (Vue 3) con el backend** que ya validaste, utilizando programación asíncrona avanzada para que la interfaz sea reactiva, robusta y profesional.



EJERCICIOS PRÁCTICOS DETALLADOS



EJERCICIO 1: Configuración del Servicio Axios Centralizado (Individual)

Objetivo: Evitar el "código espagueti" y centralizar la configuración de red.

Procedimiento Detallado:

1. En tu proyecto **erp-contable**, asegúrate de tener Axios instalado: `npm install axios`.
2. Crea la carpeta **src/services/** y dentro el archivo **erpApi.js**.
3. Implementa una instancia configurada de Axios. Debe incluir **baseURL**, **timeout** y **headers**.
4. Exporta los servicios específicos para el motor contable.

Código guía para **src/services/erpApi.js**:

```
import axios from 'axios';

const api = axios.create({
  baseURL: 'http://localhost:3000/api',
  timeout: 10000,
  headers: { 'Content-Type': 'application/json' }
});

export const movimientoService = {
  getAll: () => api.get('/movimientos'),
  create: (datos) => api.post('/movimientos', datos),
  getResumen: () => api.get('/resumen')
```

```
};  
  
export default api;
```

● EJERCICIO 2: Carga de Datos con Async/Await y Manejo de Estados (Individual)

Objetivo: Implementar el patrón profesional de carga asíncrona sin bloquear la UI.

Procedimiento Detallado:

1. Crea o modifica el componente `src/views/MotorContable.vue`.
2. Declara variables reactivas: `movimientos` (ref array), `cargando` (ref boolean), `error` (ref string).
3. Crea la función `async function cargarMovimientos()`.
4. **Regla de oro:** Debe usar `try/catch/finally`.
 - En `try`: Haz la petición con `await`.
 - En `catch`: Asigna el mensaje de error a la variable reactiva `error`.
 - En `finally`: Establece `cargando.value = false` (garantiza que el spinner se oculte aunque falle).
5. Llama a esta función dentro del hook `onMounted`.

Código guía para el `<script setup>`:

```
import { ref, onMounted } from 'vue';  
import { movimientoService } from '@/services/erpApi';  
  
const movimientos = ref([]);  
const cargando = ref(false);  
const error = ref(null);  
  
async function cargarMovimientos() {  
  cargando.value = true;  
  error.value = null;  
  
  try {  
    const respuesta = await movimientoService.getAll();  
    movimientos.value = respuesta.data.datos;  
  } catch (err) {  
    error.value = 'No se pudo conectar con el servidor. Verifica que esté  
encendido.';  
  } finally {  
    cargando.value = false;  
  }  
}  
  
onMounted(cargarMovimientos);
```

● EJERCICIO 3: Creación de Movimientos y Actualización Reactiva (Individual)

Objetivo: Enviar datos al backend y reflejar el cambio inmediatamente en la UI.

Procedimiento Detallado:

1. Agrega un formulario vinculado con `v-model` a un objeto `nuevoMovimiento`.
2. Crea la función `async function guardarMovimiento()`.
3. Usa `await movimientoService.create(...)`. Si es exitoso, haz `movimientos.value.push(respuesta.data.datos)` y limpia el formulario.
4. Maneja el error mostrando un mensaje amigable en la UI (ej. "El monto debe ser positivo" si el backend devuelve un 400).

Código guía para la función de guardado:

```
const nuevoMovimiento = ref({ concepto: '', tipo: 'Ingreso', monto: '' });

async function guardarMovimiento() {
  cargando.value = true;
  error.value = null;
  try {
    const respuesta = await movimientoService.create(nuevoMovimiento.value);
    movimientos.value.push(respuesta.data.datos); // Actualización reactiva
    nuevoMovimiento.value = { concepto: '', tipo: 'Ingreso', monto: '' }; // Limpiar
  } catch (err) {
    // Captura el mensaje específico del backend (ej: "El monto debe ser positivo")
    error.value = err.response?.data?.mensaje || 'Error al guardar el movimiento';
  } finally {
    cargando.value = false;
  }
}
```

● EJERCICIO 4: Motor Contable con Propiedades Computadas (Individual)

Objetivo: Demostrar la reactividad de Vue 3 sobre datos asíncronos.

Procedimiento Detallado:

1. Importa `computed` de `vue`.
2. Crea `totalIngresos`, `totalEgresos` y `saldo` como propiedades computadas que lean el array `movimientos`.
3. Reto: Asegúrate de que si `movimientos` está vacío, el saldo sea `0` y no `NaN`.

Código guía:

```
import { computed } from 'vue';

const totalIngresos = computed(() => {
  return movimientos.value.filter(m => m.tipo === 'Ingreso').reduce((sum, m) => sum + m.monto, 0);
});

const totalEgresos = computed(() => {
  return movimientos.value.filter(m => m.tipo === 'Egreso').reduce((sum, m) => sum + m.monto, 0);
});
```

```
});  
const saldo = computed(() => totalIngresos.value - totalEgresos.value);
```

● EJERCICIO 5: Investigación y Code Review Grupal (Pensamiento Crítico)

Objetivo: Investigar escenarios de fallo en la programación asíncrona y realizar una revisión de código cruzada.

Fase A: Investigación Individual (Entregable: `investigacion.md`)

1. **Simula un servidor caído:** Detén el servidor Express (**Ctrl + C**). Intenta cargar los movimientos en Vue. ¿Qué tipo de error se genera en la consola? ¿Cómo lo muestra tu UI?
2. **Manejo de Errores HTTP 400:** Intenta crear un movimiento con monto negativo. Atrapa el error en el `catch` y asegúrate de que tu UI muestre el mensaje exacto que envía el backend (`err.response.data.mensaje`).
3. **Investiga `Promise.all`:** Modifica tu código temporalmente para cargar `contactos` y `movimientos` simultáneamente en una sola función, esperando a que ambas terminen antes de ocultar el spinner de carga.
4. Crea un archivo `investigacion.md` con tus respuestas y capturas de pantalla.

Fase B: Code Review Grupal (Entregable: `code_review.md`)

1. **Formen parejas** e intercambien sus proyectos (o ramas de GitHub).
2. **Pruebas Cruzadas:** Ejecuta el proyecto de tu compañero. Apaga tu servidor Express y verifica que la UI de tu compañero muestre el error amigable sin colgarse.
3. **Dejen comentarios en el código** de su compañero (mínimo 3 comentarios: 1 positivo, 1 sugerencia de mejora, 1 pregunta técnica).
4. **Completen el archivo `code_review.md`** con la siguiente plantilla:

```
# Code Review - Semana 8  
  
## Revisor: [Nombre del compañero] | Revisado: [Tu nombre]  
  
### Aspectos Positivos  
1.  
2.  
  
### Sugerencias de Mejora  
1.  
2.  
  
### Preguntas Técnicas  
1.  
  
### Validación de Funcionalidades  
- [ ] Carga de movimientos funciona  
- [ ] Creación de movimientos funciona  
- [ ] Manejo de errores es adecuado (prueba con servidor apagado)
```

- [] KPIs (saldo) se actualizan correctamente
- [] El código usa async/await y try/catch/finally



PASO OBLIGATORIO: VALIDACIÓN AUTOMATIZADA

Para agilizar la corrección y darte feedback inmediato, el repositorio incluye un script de autoevaluación. **No necesitas crearlo, ya está en la raíz de `erp-contable`.**

Instrucciones:

1. Abre la terminal en la carpeta `erp-contable`.
2. Ejecuta el siguiente comando:

```
node validar_taller.js
```

3. El script analizará tu código y generará un archivo llamado `reporte_automatico.json`.
4. **Debes subir este archivo JSON** a la carpeta de Google Drive compartida junto con tu proyecto.
5. *Nota:* Si el reporte dice `"REQUIERE_REVISIÓN_MANUAL"`, lee las recomendaciones que te muestra la consola y corrige tu código antes de entregar.



INSTRUCCIONES DE ENTREGA (Google Drive)

Sube una carpeta con la siguiente estructura exacta a la carpeta compartida de Drive:

```
Apellido_Nombre_Semana8/
├── erp-contable/                # Tu proyecto de Vue completo
│   ├── src/services/erpApi.js
│   ├── src/views/MotorContable.vue
│   ├── package.json
│   └── validar_taller.js        # (Ya viene en el repo, no lo borres)
├── reporte_automatico.json     # ★ ARCHIVO CLAVE (Generado por el script)
├── investigacion.md            # 📄 Respuestas a las preguntas de la Fase A
└── code_review.md              # 👥 Plantilla completada de la Fase B
```



RÚBRICA DE EVALUACIÓN (15% Total)

A. Evaluación Individual (10%)

Criterio	Excelente (100%)	Satisfactorio (70%)	En Desarrollo (40%)	No Evidenciado (0%)	Peso
1. Arquitectura	Crea <code>src/services/erpApi.js</code>	Usa Axios, pero	Usa <code>fetch</code> en lugar de Axios sin	No hay capa de servicio,	2.5%

Criterio	Excelente (100%)	Satisfactorio (70%)	En Desarrollo (40%)	No Evidenciado (0%)	Peso
de Servicios	con <code>axios.create</code> configurado correctamente (baseUrl, headers).	configura la URL completa en cada llamada (sin instancia centralizada).	justificación, o la configuración es incorrecta.	peticiones hardcodeadas en el template.	
2. Manejo Asíncrono Robusto	Implementa <code>async/await</code> dentro de un bloque <code>try/catch/finally</code> impecable. El estado <code>cargando</code> se maneja en <code>finally</code> .	Usa <code>try/catch</code> , pero olvida el <code>finally</code> (el spinner de carga se queda pegado si hay error).	Usa <code>.then().catch()</code> de forma anidada y confusa, o no maneja errores.	Código síncrono o sin manejo de errores (la app se rompe si falla la red).	3.5%
3. Reactividad y Lógica (Computed)	Usa <code>computed</code> para calcular saldos/ingresos. Maneja correctamente el caso de array vacío (no devuelve <code>NaN</code>).	Usa <code>computed</code> , pero falla en el cálculo si el array está vacío.	Calcula los totales con un método normal (<code>function</code>) que debe ser llamado manualmente, no reactivamente.	No hay cálculo de totales o saldos.	2.5%
4. Pensamiento Investigativo	El archivo <code>investigacion.md</code> responde correctamente y demuestra manejo de error 400/500 en la UI.	Responde la pregunta, pero el manejo del error en la UI es genérico (solo muestra "Error").	No responde la pregunta investigativa o el manejo de errores es inexistente.	No entrega informe o está en blanco.	1.5%

B. Evaluación Grupal (5%)

Criterio	Excelente (100%)	Satisfactorio (70%)	En Desarrollo (40%)	No Evidenciado (0%)	Peso
5. Validación Cruzada y	El <code>code_review.md</code> está completo. Prueban el proyecto del compañero con	Prueban el proyecto, pero solo con el servidor encendido. El	No hay evidencia de que hayan probado el proyecto de otro	Trabajo puramente individual.	5.0%

Criterio	Excelente (100%)	Satisfactorio (70%)	En Desarrollo (40%)	No Evidenciado (0%)	Peso
Code Review	el servidor apagado y verifican el manejo de errores. Dejan 3 comentarios constructivos.	<code>code_review.md</code> está incompleto o superficial.	compañero. El archivo <code>code_review.md</code> no existe.		

Elaborado para el curso de Programación Web Avanzada / Arquitectura Frontend-Backend.